

Introduction to Program Synthesis (SS 2026)

Chapter 4 - Advanced Methodologies

Dr. rer. nat. Roman Kalkreuth

Chair for AI Methodology (**AIM**), Faculty of Computer Science,
RWTH Aachen University, Germany



Center for
Artificial Intelligence



Advanced Methodologies

Neural Program Synthesis vs Genetic Programming

► Genetic Programming

- ~> Gradient-free symbolic search heuristic
- ~> Direct search in symbolical spaces
- ~> Ill-conditioned, noisy search spaces
- ~> Interpretable symbolic solutions

► Neural Program Synthesis

- ~> Gradient-based search in the latent space of a neural network
- ~> Numerical, non-interpretable solutions
- ~> Turing complete programs?

Advanced Methodologies

Deep Learning

- ▶ **Artificial Neural Network (ANN)** → model of computation inspired by biological brain topology and functioning
- ▶ Set of *artificial neurons* called **perceptron** and connections that mimic synapses
 - ~> Perceptrons → **nodes**
 - ~> Synapses → **edges, weights**
- ▶ ANN → directed, weighted and acyclic graph
- ▶ Perceptron → equipped with an activation function
 - ~> Mimics firing of a neuron

Advanced Methodologies

Deep Learning

<i>Weights</i>	— Modelling of weighted transitions
<i>Bias</i>	— Shifting of transitions
<i>Backpropagation</i>	— Derivation-based update of parameters
<i>Perceptron</i>	— Artificial neuron
<i>Layer</i>	— Set of perceptrons
<i>Transition function</i>	— Calculation of the neuron's output
<i>Activation function</i>	— Decision on forward propagation (of the output)
<i>Batch</i>	— Set of input samples
<i>Epoch</i>	— Iteration of the training phase

Table: List of important terms which are commonly used in the field of deep learning

Advanced Methodologies

Deep Learning

φ — activation function

η — learning rate

β — batch size

Table: List of general symbols

Advanced Methodologies

Deep Learning

- 1943 • McCulloch, Pitts: **Electronic Brain**
- 1950 • Turing: **Learning Machine**
- 1958 • Rosenblatt: **Perceptron**
- 1969 • Minsky, Papert: **XOR Problem**
- 1982 • Hopfield: **Recurrent Neural Network**
- 1986 • Rumelhart, Hinton, Williams: **Multi-layered Perceptron, Backpropagation**
- 1989 • LeCun: **Convolutional Neural Networks**
- 1997 • Hochreiter, Schmidhuber: **Long Short Term Memory (LSTM)**
- 2006 • Hinton, Rusian: **Deep Neural Networks**
- 2012 • Ng, Dean: **Recognizing Cats on YouTube**
- 2017 • Ashish et al.: **Transformer**

Advanced Methodologies

Deep Learning

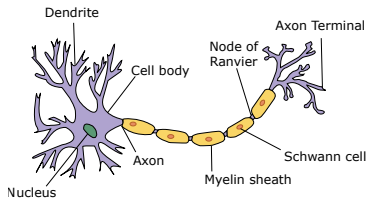


Figure: Biological neuron

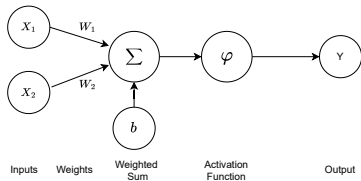


Figure: Artificial neuron (perceptron)

Advanced Methodologies

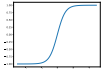
Deep Learning

Definition (Perceptron)

A perceptron $\mathcal{P}$ is a composite function g of the transition function Σ and activation function φ that receives a signal $s = (s_1, \dots, s_m)$ which is weighted by $w = (w_1, \dots, w_m)$. The function $g(\sigma \circ \varphi)$ generates a one dimensional output y .

Advanced Methodologies

Deep Learning

Identity	x		$(-\infty, \infty)$
Binary step	$\begin{cases} 0 & \text{if } x < 0 \\ 1 & \text{if } x \geq 0 \end{cases}$		$\{0, 1\}$
Logistic (sigmoid)	$\sigma(x) \doteq \frac{1}{1+e^{-x}}$		$(0, 1)$
Hyperbolic tangent (tanh)	$\tanh(x) \doteq \frac{e^x - e^{-x}}{e^x + e^{-x}}$		$(-1, 1)$
Rectified linear unit (ReLU)	$x^+ = \max(0, x) = \begin{cases} 0 & \text{if } x \leq 0 \\ x & \text{if } x > 0 \end{cases}$		$[0, \infty)$

Advanced Methodologies

Deep Learning

► Linear activation functions

- ~> Can only represent linear relationships → linear separation of data
- ~> Data from real world problems → often noisy and correlations are mostly non-linear
- ~> Used only at the output layer

► Non-linear activation functions

- ~> Enable the network to approximate a wider range of functions

Advanced Methodologies

Deep Learning

- ▶ General topology of ANN's → input, hidden and output layers
- ▶ **Single layer neural networks** → one hidden layer
 - ~ Traditional neural learning paradigm
- ▶ **Multilayer neural networks** → multiple hidden layers
 - ~ Modern deep learning approach
- ▶ Hidden layer(s) → sets of artificial neurons
- ▶ Each artificial neuron k has a transition and activation function
 - ~ $y_k = \varphi \left(\sum_{j=0}^m w_{kj} x_j \right)$
 - ~ Transition function → typically sum of the input parameters

Advanced Methodologies

Deep Learning

► General deep learning paradigm

- ~ Given a set of n samples $\mathbf{D} = \{[\mathbf{x}_1, \dots, \mathbf{x}_n], [\mathbf{y}_1, \dots, \mathbf{y}_n]\}$, find a model that approximates a function $\mathbf{f}(\mathbf{x}_i) = \mathbf{y}_i$
- ~ **Forward propagation:** Model is feed with a sample data set D where the matrix $[\mathbf{x}_1, \dots, \mathbf{x}_n]$ is the input and $[\hat{\mathbf{y}}_1, \dots, \hat{\mathbf{y}}_n]$ the output matrix
- ~ For each sample $(x_i, y_i) \in D$, the distance of the prediction to the actual is measured by the loss function $\mathcal{L}$
- ~ Parameters are adjusted in accordance to the error $\rightarrow$ **Backpropagation** via partial derivatives of the loss function
- ~ Efficient use of the chain rule $\frac{\partial z}{\partial x} = \frac{\partial z}{\partial y} \cdot \frac{\partial y}{\partial x}$

Advanced Methodologies

Deep Learning

- ▶ **Feed-forward neural network** → Commonly used type of ANN
 - ↪ Two modes with different directions of the computation flow
 - ↪ Multilayer perceptron model → **fully connected** (feed-forward) network with at least three layers
- ▶ **Forward pass:** Computation of predictions based on given observations or features
 - ↪ Forward propagation of the input data
 - ↪ Calculation of the loss $\mathcal{L}$ and cost $\mathcal{C}$
 - ↪ Quantification of the error $\mathcal{E}$ based on $\mathcal{L}$ and $\mathcal{C}$
- ▶ **Backward pass:** Backpropagation of the error
 - ↪ Derivation-based method
 - ↪ Adjustment of the parameters (weights and biases) in accordance to a learning rate η
 - ↪ Minimisation of $\mathcal{E}$

Advanced Methodologies

Deep Learning

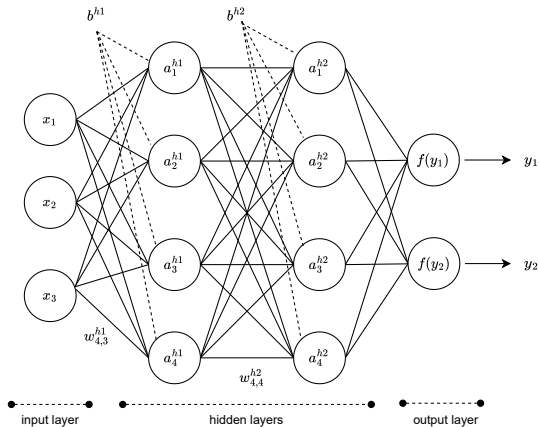


Figure: Feed-forward neural network with two hidden layers

Advanced Methodologies

Deep Learning

► Notation

- ~ $\mathbf{x}_i$ is the i -th feature of the input batch
- ~ $\hat{\mathbf{y}}_i$ is the i -th predicted output
- ~ $\mathbf{y}_i$ is the i -th desired output
- ~ $\mathbf{w}_{j,i}^t$ the weight of the connection between j -th node in layer t and i -th node in layer $t - 1$
- ~ $\mathbf{b}^t$ is the bias for the layer t
 - Global for the respective hidden layer
- ~ $\mathbf{z}^t$ is the intermediate net-input for layer t
- ~ $\mathbf{a}_i^t$ is the result of the activation function of the i -th node in layer t

Advanced Methodologies

Deep Learning: Representation

► Configuration

- ~ n : number of features (inputs)
- ~ k^t : number of nodes of layer t
- ~ l : number of hidden layers
- ~ o : number of outputs

► Representation

- ~ Weights matrix $\rightarrow \mathbf{W}$
- ~ Activation vectors $\rightarrow \mathbf{a}^t \in \mathbb{R}^{k^t}$
- ~ Biases $\rightarrow \mathbf{b} \in \mathbb{R}^l$

► Dimensions

- ~ $\dim(W^{t=1}) = k^1 \times n$ (first hidden layer)
- ~ $\dim(W^{t>1}) = k^t \times k^t$
- ~ $\dim(W^o) = o \times k$ (output layer)

$$W^{h>1} = \begin{pmatrix} w_{1,1} & w_{1,2} & \dots & w_{1,n} \\ w_{2,1} & w_{2,2} & \dots & w_{2,n} \\ \vdots & \vdots & & \vdots \\ \vdots & \vdots & & \vdots \\ w_{k,1} & w_{k,2} & \dots & w_{k,k} \end{pmatrix}$$

Advanced Methodologies

Deep Learning

- ▶ Intuition of backpropagation → analytical gradient
 - ~> **Intermediate variables** → calculated by forward propagation
 - ~> **Intermediate gradients** → used for back propagation
- ▶ Chain rule → differentiation of a composite function
 - ~> $h = f \circ g \rightarrow h'(x) = f'(g(x))g'(x)$
 - ~> Analysis how the change of the rate of a composite function is affected

Advanced Methodologies

Deep Learning

- ▶ $\varphi = \sigma(x) = \frac{1}{1+e^{-x}}$
 - ~ $\varphi' = \sigma'(x) = \sigma(x) * (1 - \sigma(x))$
- ▶ $\mathcal{L} = (\hat{y} - y)^2$
- ▶ Intermediate variables (forward pass)
 - ~ $\hat{y} = \varphi(x^T * W_1 + b_1) * (W_2 + b_2)$
 - ~ $h_1 = x^T * W_1 + b_1$
 - ~ $z_1 = \varphi(h_1)$
 - ~ $z_2 = z_1 * W_2 + b_2$
 - ~ $\mathcal{L} = (z_2 - y)^2$
- ▶ Intermediate gradients (backward pass)
 - ~ $\frac{\partial \mathcal{L}}{\partial z_2} = 2 * (z_2 - y)$
 - ~ $\frac{\partial z_2}{\partial z_1} = W_2^T$
 - ~ $\frac{\partial z_1}{\partial h_1} = \varphi'(h_1) = z_1 * (1 - z_1)$
 - ~ $\frac{\partial h_1}{\partial x} = W_1^T$

Advanced Methodologies

Deep Learning: MLP example

- ▶ Activation:

- ↪ $\varphi = \sum_{j=0}^m w_{kj} x_j$

- ▶ Forward propagation:

- ↪ $z^t = W^t * z^{t-1} + b^t$

- ↪ $a^t = \varphi(z^t)$

- ↪ $\hat{y} = a^{out}$

- ▶ Backward propagation:

- ↪ $\mathcal{L} = \frac{1}{\beta} \sum_{i=1}^{\beta} (\hat{y} - y)^2 \rightarrow \text{Mean Squared Error (MSE)}$

- ↪ $w_{i,j}^{new} = w_{i,j}^{old} - \eta \frac{\mathcal{L}}{\partial w_{i,j}}$

- ↪ $b_t^{new} = b_t^{old} - \eta \frac{\mathcal{L}}{\partial b^t}$

Advanced Methodologies

Deep Learning: MLP example

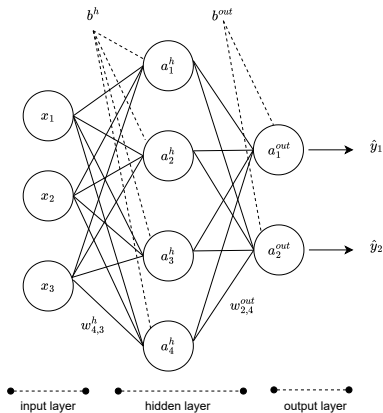


Figure: Feed-forward neural network with $n = 3$, $l = 1$, $k = 4$ and $o = 2$

Advanced Methodologies

Deep Learning: Backpropagation

- ▶ $\frac{\partial \mathcal{L}}{\partial w_{2,4}^{out}} = \frac{\partial \mathcal{L}}{\partial a_2^{out}} \frac{\partial a_2^{out}}{\partial w_{2,4}^{out}}$
- ▶ $\frac{\partial \mathcal{L}}{\partial w_{4,3}} = \left[\frac{\partial \mathcal{L}}{\partial a_1^{out}} \frac{a_1^{out}}{\partial a_4^h} \frac{a_4^h}{\partial w_{4,2}^h} + \frac{\partial \mathcal{L}}{\partial a_2^{out}} \frac{a_2^{out}}{\partial a_4^h} \frac{a_4^h}{\partial w_{4,2}^h} \right]$
- ▶ $\frac{\partial \mathcal{L}}{\partial b^{out}} = \left[\frac{\partial \mathcal{L}}{\partial a_1^{out}} \frac{\partial a_1^{out}}{\partial b^{out}} + \frac{\partial \mathcal{L}}{\partial a_2^{out}} \frac{\partial a_2^{out}}{\partial b^{out}} \right]$
- ▶ $\frac{\partial \mathcal{L}}{\partial b^h} \rightarrow$ left as exercise

Advanced Methodologies

Deep Learning: Backpropagation

- ▶ Backpropagation → commonly done with computational graphs
 - ~> Construction of a computational graph for the derivatives
 - ~> Used for symbolic and automatic differentiation¹
- ▶ Symbol-to-Symbol derivatives → approach to computing partial derivatives
 - ~> Optimisation of the computational load needed for training of ANN's
- ▶ Computational graph is build to calculate derivatives
 - ~> Efficient re-use of partial derivatives

Advanced Methodologies

Deep Learning: Backpropagation

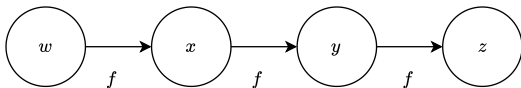


Figure: Computational graph consisting of subexpressions obtained when computing the gradient

$$\begin{aligned}\frac{\partial w}{\partial z} &= \frac{\partial z}{\partial y} \frac{\partial y}{\partial x} \frac{\partial x}{\partial w} \\ &= f'(y) f'(x) f'(w) \\ &= f'(f(f(w))) f'(f(w)) (f'(w))\end{aligned}\tag{1}$$

Advanced Methodologies

Deep Learning: Backpropagation

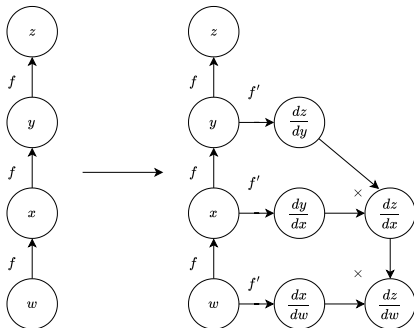
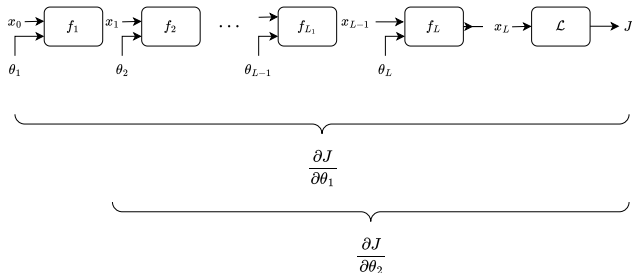


Figure: Symbol-to-Symbol approach for backpropagation

Advanced Methodologies

Deep Learning: Backpropagation



- Simple computational graph $\rightarrow$ chain of functions
 - $\rightsquigarrow f_L \circ f_{L-1} \circ \dots \circ f_2 \circ f_1$
 - $\rightsquigarrow$ Can be used to model the forward propagation

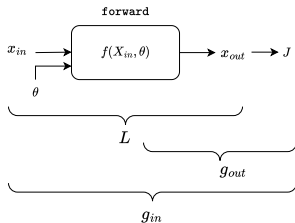
Advanced Methodologies

Deep Learning: Backpropagation

- ▶ Goal of backpropagation $\rightarrow$ minimisation of the loss J
 - $\leadsto$ Update of the parameters θ
- ▶ Calculation of gradients via chain rule
 - $\leadsto \frac{\partial J}{\partial \theta_1} := \frac{\partial J}{\partial x_L} \frac{\partial x_L}{\partial x_{L-1}} \cdots \frac{\partial x_2}{\partial x_1} \frac{\partial x_1}{\partial \theta_1}$
 - $\leadsto \frac{\partial J}{\partial \theta_2} := \frac{\partial J}{\partial x_L} \frac{\partial x_L}{\partial x_{L-1}} \cdots \frac{\partial x_2}{\partial \theta_2}$
- ▶ More efficient approach $\rightarrow$ re-use of gradient computation

Advanced Methodologies

Deep Learning: Backpropagation



- ▶ $L, g \rightarrow$ partial derivatives of a layer
- ▶ Computing L is based on the derivative of the transition function $f'(x_{in}, \theta)$
- ▶ Computation of g is done by evaluating the chain rule layer-wise

Advanced Methodologies

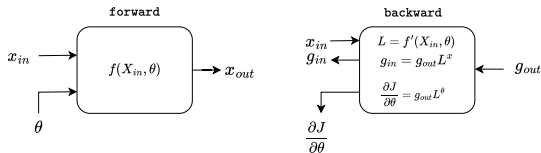
Deep Learning: Backpropagation

- ▶ $g_l := \frac{\partial J}{\partial x_l} \triangleleft$ gradient of the loss w.r.t x_l
- ▶ $L := \frac{\partial x_{out}}{\partial [x_{in}, \theta^t]} \triangleleft$ gradient of the layer
 - $\leadsto L^x := \frac{\partial x_{out}}{\partial x_{in}} \triangleleft$ w.r.t input data of the layer
 - $\leadsto L^{\theta^t} := \frac{\partial x_{out}}{\partial \theta^t} \triangleleft$ w.r.t parameters of the layer

- ▶ $\frac{\partial J}{\partial \theta^t} := \overbrace{\frac{\partial J}{\partial x_{out}}}^{g_{out}} \overbrace{\frac{\partial x_{out}}{\partial \theta}}^{L^{\theta}} = g_{out} L^{\theta^t} \triangleleft$ application of the chain-rule
- ▶ $\theta^{t+1} \leftarrow \theta^t - \eta \left(\frac{\partial J}{\partial \theta^t} \right)^T \triangleleft$ parameter update based on gradient
- ▶ $g_{in} := g_{out} L^x \triangleleft$ backpropagation of errors through layers

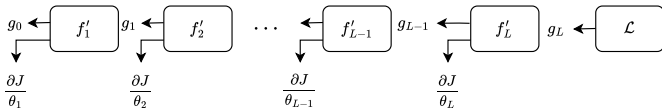
Advanced Methodologies

Deep Learning: Backpropagation



- Backpropagation is divided into two modes → forward and backward pass
 - ~ forward → layer-wise computation of immediate values
 - ~ backward → layer-wise computation of immediate gradients

Advanced Methodologies



- ▶ Full backpropagation algorithm → forward and backward pass through all layers

Advanced Methodologies

Deep Learning: Backpropagation

Algorithm Simple backpropagation: forward pass

Arguments

Parameter vector $\theta = \{\theta_l\}_{l=1}^L$

Layer transition functions $f_1, \dots, f_L$

Datapoint (x^0, y^0)

Return

Intermediate values $\mathcal{X} = \{x_l\}_{l=1}^L$

- 1: **for** $l \leftarrow 1$ to L **do**
- 2: $x_l \leftarrow f_l(x_{l-1}, \theta_l)$
- 3: **end for**
- 4: **return** $\mathcal{X}$



Advanced Methodologies

Deep Learning: Backpropagation

Algorithm Simple backpropagation: backward pass

Arguments

Parameter vector $\theta = \{\theta_l\}_{l=1}^L$

Intermediate values $\mathcal{X} = \{x_l\}_{l=1}^L$

Layer transition derivatives $f'_1, \dots, f'_L$

Loss function $\mathcal{L} : \mathcal{R}^N \rightarrow \mathcal{R}$

Return

Direction of gradient $\frac{\partial J}{\partial \theta} = \{\frac{\partial J}{\partial \theta_l}\}_{l=1}^L$

```
1:  $g_L = \mathcal{L}'(x_L, y)$ 
2: for  $l \leftarrow L$  to 1 do
3:    $L_l \leftarrow f'_l(x_{l-1}, \theta_l)$ 
4:    $g_{l-1} \leftarrow g_l L_l^x$ 
5:    $\frac{\partial J}{\partial \theta_l} \leftarrow g_l L_l^\theta$ 
6: end for
```

Advanced Methodologies

Deep Learning: Backpropagation

- ▶ State-of-the-art deep learning → stochastic gradient descent (SGD)²
 - ↪ *Minibatch* of examples is sampled from the training dataset
 - ↪ Computation of an estimate of the gradient
- ▶ Reduces the computational cost of the training
 - ↪ The exact gradient is expensive to compute
 - ↪ Approximation of the gradient is sufficient for training

Advanced Methodologies

Deep Learning: Backpropagation

Algorithm Stochastic gradient descent (SGD)

Arguments

Learning rate η

Initial parameters θ^0

Training set $\{(x^1, y^1), \dots, (x^n, y^n)\}$

- 1: **while** stopping criterion not met **do**
 - 2: Sample m examples from the training set $\{(x^1, y^1), \dots, (x^m, y^m)\}$
 - 3: Compute gradient estimate: $\hat{g} \leftarrow +\frac{1}{m} \nabla_{\theta} \sum_i \mathcal{L}(f(x^i, \theta), y^i)$
 - 4: Apply update: $\theta \leftarrow -\eta \hat{g}$
 - 5: **end while**
-